Linux Fundamentals and Command-Line Navigation

Master the operating system powering the cloud. Navigate filesystems, manage files, inspect text output, and operate the shell with confidence.

DAY LEARNING OBJECTIVES

- **Objective:** Explain why Linux is the dominant operating system for cloud infrastructure, containers, and web servers
- **Objective:** Understand the distinction between the operating system kernel, the terminal emulator, and the shell interpreter
- **Objective:** Navigate the Linux Filesystem Hierarchy Standard (FHS) using absolute and relative paths
- **Objective:** Identify the core system directories: /home, /etc, /var, /tmp, /usr, /opt, and /var/log
- **Objective:** Create, inspect, copy, move, and remove files safely in a simulated terminal environment
- **Objective:** Read files efficiently using cat, head, and tail, and extract help information using --help and man

Topic 1: Linux and Common Distributions (20 mins)

Over 90% of all public cloud workloads, 100% of the world's top 500 supercomputers, and virtually all Docker container images run on Linux. Understanding Linux is not optional for a DevOps engineer—it is your everyday working environment.

What is Linux?:
Technically, "Linux" is not a complete operating system; it is an open-source monolithic kernel created by Linus Torvalds in 1991. The kernel is the core software layer that manages hardware interactions: CPU scheduling, RAM allocation, disk I/O, and network device drivers.

A Linux Distribution (Distro) pairs the Linux kernel with GNU userland utilities, package managers, system libraries, and background service managers (like systemd) to create a complete, installable operating system.

The Major Distribution Families:

1. Debian Family (Ubuntu, Debian):
   - Package Manager: apt / dpkg (deb packages).
   - Highly popular in developer environments, cloud VMs, and default container images. Ubuntu Server is the most widely deployed cloud OS on AWS and GCP.
2. Red Hat Family (RHEL, Rocky Linux, AlmaLinux, Fedora):
   - Package Manager: dnf / yum / rpm.
   - The corporate enterprise standard across banking, government, and legacy corporate IT. Renowned for rock-solid stability and SELinux enterprise security.
3. Alpine Linux:
   - Package Manager: apk.
   - Ultra-minimal distribution (~5 MB base footprint) built around musl libc and busybox. Widely chosen for production Docker microservices to minimize attack surface and download times.

- **Takeaway:** Linux is the standard OS for cloud computing, microservices, and containerized workloads.
- **Takeaway:** Ubuntu/Debian uses apt; RHEL/Rocky uses dnf/rpm; Alpine uses apk.
- **Takeaway:** The kernel communicates with physical hardware; the userland provides software tools.

Topic 2: Operating System, Terminal, and Shell (20 mins)

Beginners frequently confuse the terms Console, Terminal, and Shell. Distinguishing them gives you a clear mental model of how commands execute.

1. The Terminal (Terminal Emulator):
The terminal is the graphical window program that accepts keyboard input and renders text output on screen (e.g. iTerm2 on Mac, Windows Terminal, GNOME Terminal, or your cloud web console). It does not execute commands itself; it simply routes your keystrokes to the shell.

2. The Shell (Command Interpreter):
The shell is the command-line interpreter that reads your commands, interprets what you typed, evaluates environment variables and wildcards, and requests the Linux kernel to launch the appropriate executable programs.
- Bash (Bourne Again Shell): The default standard shell on almost all Linux servers. Scripts typically begin with the #!/bin/bash shebang.
- Zsh (Z Shell): Default on modern macOS with rich autocomplete and themes.
- sh (Bourne Shell): Minimal POSIX standard shell found on embedded systems and Alpine containers.

The Shell Command Anatomy:
A shell command typically follows this structure:

```
command  [options/flags]  [arguments]
ls       -l               /var/log
```

`
- command: The executable binary or built-in function to run (ls).
- options/flags: Modifies command behavior. Usually preceded by a single dash for short flags (-l, -a) or two dashes for long flags (--all).
- arguments: The target resource or data the command operates upon (/var/log`).

- **Takeaway:** The terminal is the display window; the shell is the program that interprets and runs commands.
- **Takeaway:** Bash is the universal standard for Linux servers and CI/CD automation pipelines.
- **Takeaway:** Commands consist of an executable name, optional modifier flags, and target arguments.

Topic 3: Filesystem Structure (FHS) (25 mins)

Unlike Windows, which organizes disks into separate drive letters (C:\, D:\), Linux organizes everything into a single unified hierarchical tree starting at the root directory: / (forward slash).

In Linux, there is a fundamental architectural rule: "Everything is a file". Hard disks, memory states, network sockets, keyboard inputs, and running process information are all exposed through the filesystem.

The Linux Filesystem Hierarchy Standard (FHS):

``
/ (Root)
% % % home/   # Personal directories for non-root users (e.g., /home/devops)
% % % root/   # Home directory of the superuser (root)
% % % etc/   # System-wide configuration files (e.g., /etc/nginx, /etc/hosts)
% % % var/   # Variable, changing data (e.g., /var/log, /var/www, /var/run)
% % % tmp/   # Temporary files (cleared automatically on reboot)
% % % usr/   # User binaries and shared read-only software libraries
%   % % % bin/   # Standard user commands (e.g., /usr/bin/python3, /usr/bin/git)
% % % bin/   # Essential user binaries (symlinked to /usr/bin on modern distros)
% % % opt/   # Optional third-party software packages (e.g., /opt/datadog)
% % % proc/   # Virtual filesystem exposing kernel & process state in RAM
% % % dev/   # Device files representing physical and virtual hardware
`

Directories Every DevOps Engineer Must Know::
- /etc: Where application configurations live. When you configure Nginx, SSH, or PostgreSQL, your config files are in /etc.
- /var/log: Where system and application log files are stored. The first place you look when troubleshooting an outage.
- /tmp`: Used by build scripts and scrapers for transient files. Never store persistent data here because it can be wiped at any time.

- **Takeaway:** Linux uses a single unified tree starting at the root slash (/).
- **Takeaway:** System and service configurations live in /etc; runtime logs live in /var/log.
- **Takeaway:** /tmp is temporary and can be wiped on system reboot.

Topic 4: Navigation Commands: pwd, ls, and cd (25 mins)

When you connect to a Linux server via SSH, there is no graphical mouse or desktop. You navigate by changing your Current Working Directory (CWD).

1. pwd (Print Working Directory):
Prints the full absolute path of the directory you are currently in.
``bash
$ pwd
/home/devops/workspace
`

2. ls (List Files and Folders):
Displays the contents of a directory. Essential flags:
- ls: Basic list of visible files.
- ls -l: Long listing format showing permissions, owner, group, file size in bytes, and timestamp.
- ls -a: Shows all files, including hidden dotfiles (files starting with a . like .env or .git).
- ls -lh: Long listing with human-readable file sizes (KB, MB, GB instead of raw bytes).
- ls -lt: Sorts files by modification time, newest first (invaluable for spotting recent log outputs).

3. cd (Change Directory):
Moves your shell to a different directory location.
- Absolute Paths: Start from root / (e.g. cd /var/log/nginx). Works from anywhere.
- Relative Paths: Start relative to your current location:
  - . (single dot): Current directory.
  - .. (double dot): Parent directory (one level up).

- ~ (tilde): Current user's home directory (/home/username).
- cd -`: Toggles back to the previous directory you were in before your last move.

- **Takeaway:** pwd tells you where you are; ls shows what is there; cd moves you somewhere else.
- **Takeaway:** Use ls -lah to see all files (including hidden .env files) with readable sizes.
- **Takeaway:** cd .. moves one level up; cd ~ takes you directly home.

Topic 5: File and Directory Operations: mkdir, touch, cp, mv, and rm (25 mins)

Creating, duplicating, moving, and safely deleting files are the core mechanics of managing application deployments.

1. mkdir (Make Directory):
Creates a new directory.
``bash
$ mkdir project_logs
# Use -p (parents) to create nested directory paths in one command without errors:
$ mkdir -p app/backend/config
`

2. touch (Create Empty File or Update Timestamp):
Quickly creates a blank, empty file if it does not exist, or updates its last-modified timestamp if it already exists.
`bash
$ touch .env.example
$ touch app.log
`

3. cp (Copy Files and Folders):
Copies files or entire directory trees from a source to a destination.
`bash
# Copy a single file:
$ cp config.example.json config.json

# Copy an entire directory recursively using -r:
$ cp -r src/ backups/src_backup/
`

4. mv (Move or Rename Files):
Moves a file to a new location OR renames it if the path is in the same directory.
`bash
# Renaming a file:
$ mv server.js index.js

# Moving a file into another folder:
$ mv index.js src/
`

5. rm (Remove Files and Folders):
Permanently deletes files from the filesystem. WARNING: Linux has no "Recycle Bin" or Trash can. Once rm executes, the file is unlinked immediately.
- rm app.log: Removes a single file.
- rm -r folder/: Recursively deletes a directory and all of its contents.
- rm -f: Force removal without confirmation prompts.
- DANGER: rm -rf / or rm -rf /* will delete the entire operating system! Never run this command. Always double-check your current path with pwd before using rm -rf`.

- **Takeaway:** mkdir -p creates nested directory trees safely.
- **Takeaway:** cp requires the -r flag to copy directories; mv renames or relocates files.
- **Takeaway:** rm has no undo or recycle bin; exercise extreme caution with rm -rf.

Topic 6: Reading Files and Inspecting Output: cat, head, and tail (20 mins)

DevOps engineers frequently inspect configuration files, environment definitions, and application log output. Choosing the right command prevents flooding your terminal screen.

1. cat (Concatenate and Display):
Prints the entire contents of a file to the terminal output.
``bash
$ cat /etc/hosts
127.0.0.1   localhost
`

Best for: Short configuration files (under 50 lines). If you run cat on a 500 MB log file, your terminal will freeze as millions of lines flash across the screen!

2. head (View Top of File):
Prints the first few lines of a file (default is 10 lines).
`bash
# View the first 5 lines of a log:
$ head -n 5 /var/log/syslog
`

3. tail (View Bottom of File):
Prints the last few lines of a file (default is 10 lines).
`bash
# View the last 15 lines of an application log:
$ tail -n 15 app.log

# Stream live updates in real time using -f (follow):
$ tail -f /var/log/nginx/access.log
`

The tail -f command is one of the most frequently used diagnostic commands in DevOps. It keeps the file connection open and prints new log entries as they are written by the web server in real time. Press Ctrl+C` to stop streaming.

- **Takeaway:** cat is for short files; do not use it on massive logs.
- **Takeaway:** head shows the top of a file; tail shows the bottom.
- **Takeaway:** tail -f follows log updates live in real-time, crucial for testing running services.

Topic 7: Command Arguments and Help: --help and man (20 mins)

No engineer memorizes every single flag for every Linux command. Senior engineers are simply faster at looking up help information in the terminal.

1. The --help Flag:
Most modern command-line utilities provide a built-in help overview when passed --help or -h:
``bash
$ ls --help
$ docker --help
$ git checkout --help
`

This displays accepted arguments, short/long flags, and brief descriptions directly in your terminal.

2. man (Manual Pages):
The man command displays the official system manual for a tool:
`bash
$ man ls
$ man ps
`

This opens a scrollable, searchable pager (typically using less):
- Press Space or Down Arrow to scroll down.
- Press b or Up Arrow to scroll up.
- Type /<keyword>` and press Enter to search for a word (press n for next match).
- Press q to quit and return to your shell prompt.

- **Takeaway:** Never guess flags; use --help or man to inspect available options.
- **Takeaway:** man opens a scrollable manual; press q to exit the pager.
- **Takeaway:** Search within man pages by typing / followed by your search keyword.

INTERVIEW REVISION VAULT (TOP 5 QUESTIONS)

Q1: Why do containers (like Docker) predominantly use Alpine or Debian Linux instead of Windows or macOS as base images? [Target Role: DevOps / Cloud Engineer | Beginner]

Model Answer: Linux is open-source, lightweight, and built with native kernel isolation primitives (namespaces and cgroups) that make container virtualization possible without hypervisor overhead. Distributions like Alpine (~5 MB) or Debian Slim (~70 MB) allow container images to download in seconds, consume minimal RAM, have zero licensing costs, and minimize attack surfaces by excluding unnecessary desktop packages.

Key Evaluation Signals: Linux has native kernel cgroups and namespaces for process isolation. • Alpine/Debian provide tiny image footprints without licensing costs. • Minimal base images reduce security vulnerability attack surface.

Common Candidate Pitfalls: Saying Windows cannot run containers (Windows containers exist, but are heavy and costly). • Failing to mention namespaces and cgroups.

Q2: Explain the difference between a hard link and a symbolic (soft) link in the Linux filesystem. [Target Role: Linux / Systems Administrator | Intermediate]

Model Answer: A hard link is an additional directory entry pointing directly to the exact same inode (physical data blocks on disk). If you delete the original file, the hard link still accesses the data. Hard links cannot span across different disk partitions or point to directories. A symbolic link (symlink, created with ln -s) is a special pointer file containing a path string referencing another target file. If the target file is deleted or moved, the symlink breaks ("dangling symlink"). Symlinks can span different disks and point to directories.

Key Evaluation Signals: Hard link shares the same inode; deleting original does not delete data. • Soft/symlink stores the target path; breaks if target is removed. • Symlinks can cross filesystem partitions; hard links cannot.

Common Candidate Pitfalls: Claiming hard links and symlinks are identical. • Not knowing what an inode is.

Q3: A junior engineer ran "cat production.log" on an active 10 GB log file and the terminal froze. How do you recover and what should they have done instead? [Target Role: DevOps / SRE | Scenario]

Model Answer: To recover immediately, press Ctrl+C to send SIGINT. If the terminal buffer is completely locked up, open a separate terminal window and terminate the cat process using pkill -9 cat. To inspect large log files safely, you should never use cat. Instead: 1) Use tail -n 100 to inspect the most recent entries, 2) Use tail -f to stream live updates, 3) Use less production.log to view the file page-by-page with on-demand streaming without loading the entire 10 GB into memory, or 4) Use grep to filter for specific error timestamps.

Key Evaluation Signals: Terminate with Ctrl+C or kill the process from another session. • Use less for memory-efficient paginated navigation. • Use tail -n or tail -f for inspecting recent entries.

Common Candidate Pitfalls: Suggesting to reboot the production server. • Recommending downloading the 10 GB file to a laptop.

Q4: What is the purpose of the /etc, /var, and /tmp directories in the Linux Filesystem Hierarchy Standard? [Target Role: Junior DevOps Engineer | Beginner]

Model Answer: /etc is reserved for static, host-specific system and application configuration files (e.g. /etc/nginx/nginx.conf, /etc/fstab). /var contains variable runtime data that changes frequently while the system is running (e.g. log files in /var/log, package caches, and database data). /tmp is used for temporary, transient scratch files created by running programs, and its contents are typically wiped automatically upon system reboot or periodically by systemd-tmpfiles.

Key Evaluation Signals: /etc for configuration files. • /var for runtime variable data and logs. • /tmp for temporary files wiped on reboot.

Common Candidate Pitfalls: Storing persistent backups or databases in /tmp. • Confusing /etc with binary executable directories.

Q5: How do you check available disk space and determine which directory is consuming the most storage on a Linux server? [Target Role: Infrastructure / Systems Engineer | Intermediate]

Model Answer: First, I run df -h (disk free, human-readable) to identify which mounted filesystem partition is filling up (e.g. if /var is at 98%). Next, I navigate to the problem partition and run du -sh * | sort -hr | head -n 10 (disk usage, summary, human-readable, sorted in reverse order) to quickly pinpoint the top 10 largest subdirectories. If a log file is the culprit, I can truncate it safely using truncate -s 0 file.log rather than deleting an open file descriptor.

Key Evaluation Signals: df -h shows filesystem partition percentages. • du -sh * | sort -hr reveals directory size consumption. • Truncating open logs prevents "deleted but still consuming disk" descriptor leaks.

Common Candidate Pitfalls: Confusing df (disk free) with du (disk usage). • Deleting an active log file that is held open by a running process, failing to reclaim space.

DAY SUMMARY & RAPID REVISION CHECKLIST

- **Verified Concept:** Linux is the dominant cloud operating system; Debian uses apt, Red Hat uses dnf, Alpine uses apk.
- **Verified Concept:** FHS standard: /etc contains configurations, /var/log contains logs, /tmp contains temporary files.
- **Verified Concept:** pwd prints working directory, ls -lah displays all files with human-readable sizes, cd .. moves up.
- **Verified Concept:** mkdir -p creates nested parents; cp -r recursively copies directories; mv renames or moves.
- **Verified Concept:** Never run rm -rf without verifying your current directory using pwd; Linux has no Recycle Bin.
- **Verified Concept:** Use cat for small files; use tail -n or tail -f to stream active server logs safely without memory lockup.